

MID-STATE TECHNICAL COLLEGE • IT PROGRAMS

Users, Groups, and Authentication

Unit 3 • Linux (10-151-105)



Learning Outcomes

By the end of this unit, you will be able to:



Manage user accounts

Create, modify, remove, and troubleshoot accounts and their configuration files.



Manage group accounts

Organize users into groups and script repetitive account tasks.



Modify user configurations

Customize shells, variables, aliases, and password aging.



Escalate privileges

Elevate safely with su and sudo under the principle of least privilege.

LESSON 1

Managing User Accounts



What Is a User Account?

- An account is an identity the system uses to grant access to files, applications, and resources.
- Linux tracks accounts by a numeric **user ID (UID)** — not by the username.
- The username (e.g. `kgarcia`) is for humans; the UID is for the system.
- An **Effective UID (EUID)** is used temporarily during privilege escalation or when running as another user.



Permissions answer one question

“Does this UID have the right to use this resource?”

Get identity right and access control becomes possible. Get it wrong and nothing above it can be trusted.

Three Types of Accounts

Every account lives in the same file but serves a very different role:

Account type	Purpose	Typical UID	Can log in?
root	The superuser — complete control of the system	0	Yes (discouraged)
System / service	Run the OS and its services (web, mail, etc.)	1–999	No
Standard user	Real people who authenticate to use resources	1000 and up	Yes

Exact ranges vary by distribution — /etc/login.defs defines UID_MIN and UID_MAX.

Where Accounts Live: /etc/passwd

Each line is seven colon-separated fields:

```
kgarcia:x:1001:1001:Kai Garcia:/home/kgarcia:/bin/bash
```

- **kgarcia** — username used to log in
- **x** — placeholder; the real hash is in /etc/shadow
- **1001** — user ID (UID)
- **1001** — primary group ID (GID)
- **Kai Garcia** — comment / GECOS (full name)
- **/home/kgarcia** — home directory
- **/bin/bash** — login shell

Never hand-edit these files — a stray typo can lock everyone out. Use the account commands.

Passwords Live in `/etc/shadow`

- Password hashes — and password-aging data — live in `/etc/shadow`, readable only by root.
- Originally passwords sat in `/etc/passwd`, which is world-readable — so every user could read them.
- Moving the hashes to a root-only file closed that hole.
- `/etc/shadow` also stores last-change, expiration, and warning fields (managed with `chage`).



SECURITY FOCUS

Separating secrets from the world-readable account file is least privilege in miniature: expose only what must be exposed, and protect the rest behind root.

Why it matters: A world-readable password file would hand every credential on the system to any logged-in user.

Creating Users: useradd

The low-level account creator on every distribution. Syntax: `useradd [options] username`

Option	What it does
-c	Comment / full name (quote multi-word values)
-m	Create the home directory
-s	Set the login shell
-e	Set an expiration date (YYYY-MM-DD)
-u	Assign a specific UID

Example

```
useradd -c "Kai Garcia" \  
-m -s /bin/bash kgarcia
```

useradd creates the account but does NOT set a password — the account can't log in yet.

LIVE DEMO

Create a user on Ubuntu, then `tail /etc/passwd` to confirm the new line.

Creating Users: adduser (Debian)

Ubuntu and other Debian-family systems add a friendlier, interactive front end.

useradd

- Low-level, scriptable, identical syntax across distros
- No prompts — you supply every option
- Does not set a password or build the home dir unless told

adduser

- Interactive — walks you through each field
- Prompts you to set the password
- Creates the home directory by default

Dual-ecosystem takeaway: learn `useradd` for portability; reach for `adduser` on Ubuntu when you want the guided, password-included flow.

Setting & Viewing Accounts

Set passwords

- `passwd kgarcia` — set/reset a user's password
- No characters echo while typing — by design
- `passwd` alone changes your own password

Inspect accounts

- `id kgarcia` — UID, GID, and group memberships
- `getent passwd` — local + directory (LDAP) accounts
- `whoami` — your current username
- `w / who` — who is logged in right now

LIVE DEMO

Run `id` and `getent passwd` against a freshly created account.

Modifying Users: usermod

Change an existing account without recreating it. Syntax: `usermod [options] username`

Option	What it does
-c	Change the comment / full name
-s	Change the login shell
-e	Set or change the expiration date
-l	Rename the account (login name)
-aG	Append the user to a supplementary group

Locked & expired

Setting an expiration with `-e` does not delete the account — it just stops it from authenticating after that date.

Neither `usermod` nor `userdel` will change a user that has running processes.

LIVE DEMO

Rename a user with `-l`, then confirm with `id`.

Removing Users: `userdel` & `deluser`

`userdel` (all distros)

- `userdel kgarcia` — remove the account, keep the home dir
- `userdel -r kgarcia` — also remove the home directory

`deluser` (Debian)

- Friendly wrapper; can back up files first
- `--remove-home` / `--backup-to`
- Can run custom pre-delete scripts

Think before you delete. Deleted accounts can't be recovered — recreating makes a new UID, a new identity. Home directories may hold needed or proprietary data, so back up first. Often it's better to **disable** an account (`passwd -l` or `usermod -L`) than remove it.

Did It Work? Exit Codes

- On success, the account commands print **nothing** — silence is good.
- `echo $?` shows the exit code of the last command.
- **0** means success; any other value is an error.

Codes differ slightly across `useradd`, `usermod`, and `userdel` — check the man page for each.

Code	Meaning (useradd)
0	Success
1	Couldn't update /etc/passwd
9	Username already in use
12	Couldn't create the home directory

Tracking Logins & Troubleshooting

See who logged in

- `last` — every login/logout (from `/var/log/wtmp`)
- `lastlog` — most recent login per account
- `faillock` — failed attempts (`pam_faillock` can lock accounts)

Can't log in? Check in order

1. Does the account exist?

(check `/etc/passwd`)

2. Is a password set?

(check `/etc/shadow`)

3. Is it locked or expired?

(`passwd -u`, `usermod -U`, `chage`)

LESSON 2

Managing Group Accounts



Why Groups?

- A group is a collection of accounts with similar access needs.
- Grant a permission to one group of five instead of to five users individually — less work, fewer mistakes, consistent access.
- Groups live in `/etc/group`, each with a name and a numeric **GID**.
- Every user has one **primary group** and may belong to many **supplementary groups**.



Primary vs supplementary

Primary group is recorded in `/etc/passwd` (the GID field).

Supplementary groups are recorded in `/etc/group`.

Group Commands

Groups follow the same create / modify / delete lifecycle as users.

Command	What it does
groupadd sales	Create a new group
groupmod -n publicity marketing	Rename a group (-n)
groupdel sales	Delete a group
addgroup / delgroup	Friendly Debian wrappers

Remember

Deleting a group does not delete its users — a group is a separate object from the accounts in it.

LIVE DEMO

Create a group, rename it with -n, then tail /etc/group.

Adding Users to Groups

Add a member with **usermod -aG** — and the **-a** is not optional.

Correct

```
usermod -aG sales jdeng
```

Appends sales, keeps all existing memberships.

Dangerous

```
usermod -G sales jdeng
```

Replaces ALL supplementary groups with just sales.

- **-a** appends; **-G** alone overwrites the supplementary group list.
- Adding a member modifies the user record, not the group.
- Dropping -a can silently strip someone's access — including admin group membership.

LIVE DEMO

Add a user to a group with -aG, then confirm with id.

Group Gotchas & Exit Codes

- You can't delete a user's **primary group** until the user is removed.
- Before deleting a group, reassign any files or resources it owns.
- A delete blocked by running processes? Stop them: `sudo killall -u username`

Code	Meaning (groupadd)
0	Success
4	GID not unique
9	Group name not unique

Scripting tip: a short Bash script can create groups and add members the same way every time — consistent, fast, and typo-free.

LESSON 3

Modifying User Configurations



Customizing the Shell Prompt

- The prompt is the `PS1` variable, set in `~/.bashrc`.
- Change the default shell yourself with `chsh -s /bin/bash` (must be listed in `/etc/shells`).

```
PS1="\u@\h:\w\$ "
```

→ *user@host:dir\$*

Code	Shows
\u	Username
\h	Hostname
\w	Current directory
\\$	# for root, \$ for users
\t \d	Time / date

Profile & Startup Files

System-wide (all users)

- `/etc/profile` — environment + startup for login shells
- `/etc/bashrc` — system-wide functions and aliases
- `/etc/skel` — template copied into every new home dir

Per-user (overrides system)

- `~/.bash_profile` — login-shell settings
- `~/.bashrc` — interactive non-login shells
- `~/.profile` — applies to local and SSH logins

Precedence: the system files load first, then the user files — so a user's settings override the system defaults. Drop files in `/etc/skel` to pre-populate every new account.

Shell vs. Environment Variables

Shell (local) variables

- Live only in the current shell
- Not passed to child processes
- Declare with `name=value` (no spaces around =)

Environment variables

- Inherited by child processes
- Promote with `export NAME=value`
- Common: `HOME`, `USER`, `PATH`, `SHELL`

```
my_name=kgarcia then echo $my_name → kgarcia
```

Reference a variable's value by prefixing its name with \$.

The PATH Variable

- **PATH** is the colon-separated list of directories the shell searches for commands.
- Add a directory so its programs run by name; separate entries with `:`.
- Run a program that isn't on PATH by prefixing `./` (search the current directory).



SECURITY FOCUS

Never put your home directory (or `./`) on PATH. A malicious file named like a common command could then run automatically instead of the real tool.

Why it matters: PATH order decides which program runs — an attacker who controls an early directory controls your commands.

Aliases & Command History

Aliases

- Shorthand for longer commands: `alias ll="ls -alh"`
- `alias` lists them; `unalias` removes one
- Persist them in `~/.bashrc` or `~/.bash_aliases`

History

- `HISTSIZE` — commands kept in memory
- `HISTFILESIZE` — lines saved across reboots
- `HISTCONTROL=ignoredups` — drop duplicates

Aliases point to a modified command; variables store a value — similar look, different jobs.

Password Policy: login.defs & chage

System defaults — /etc/login.defs

- `PASS_MAX_DAYS` — max age before change
- `PASS_MIN_DAYS` — min wait before change
- `PASS_WARN_AGE` — warning window
- `PASS_MIN_LEN` — minimum length

Per-account aging — chage

- `-l` — show an account's settings
- `-M / -m` — max / min days
- `-W` — warning days
- `-E` — account expiration date

login.defs sets new-password defaults; chage and passwd adjust existing accounts.

LIVE DEMO

Run `chage -l` on an account, then set a 90-day max with `chage -M 90`.

LESSON 4

Escalating Privileges



Least Privilege & the Root Account

- **root** (UID 0) can do anything — necessary, but dangerous for daily use.
- Log in with a standard account; elevate only for the task that needs it.
- This is the **principle of least privilege**: grant the minimum access required.
- Many distros (including Ubuntu) disable direct root login and expect admins to use sudo.



SECURITY FOCUS

“Get root” is shorthand for gaining admin rights. The goal is to do it briefly and deliberately — not to live there.

Why it matters: Every minute spent as root is a minute a mistake or a hijacked process runs with unlimited power.

Switching Users: su vs. su -

Both switch identity; the dash decides whether you also load their environment.

	su username	su - username
Identity	Switches to the user	Switches to the user
Environment	Keeps YOUR profile & vars	Loads THEIR login environment
Directory	Stays in your home dir	Moves to their home dir
Best for	Quick checks	A real working session

Mind the spaces: su - username has a space on BOTH sides of the dash. With no name, su - means root.

(You must know the target user's password to switch with su.)

Delegating with sudo

- Run specific commands with elevated rights — without sharing the root password.
- sudo uses **your own** password and logs every command it runs.
- `sudo -l` — list what you're allowed to run.
- Forgot to type sudo? `sudo !!` reruns the previous command with sudo.



Why sudo wins

- Per-command, not all-or-nothing
- Your password, not root's
- An audit trail of who did what
- Time-limited and configurable

Editing sudoers with visudo

- Delegation rules live in `/etc/sudoers` — never edit it directly.
- Use **visudo**: it checks syntax before saving, so a typo can't lock everyone out.
- Drop-in files in `/etc/sudoers.d/` add rules safely without touching the main file.
- Need to edit a root-only file? `sudoedit` opens it with your own credentials.

sudoers entry syntax

```
user host=(runas) command
```

```
kgarcia ALL=(root) /sbin/shutdown
```

```
Aliases group things: User_Alias,  
Cmnd_Alias, Host_Alias, Runas_Alias.
```

LIVE DEMO

Open visudo, grant a user one command, and test it.

wheel vs. sudo: The Group That Grants Power

Administrative access comes from membership in one group — and its name differs by family.

	Rocky Linux 10	Ubuntu Server 24.04
Family	Red Hat (RHEL-style)	Debian-style
Admin group	wheel	sudo
Root login	Enabled, but discouraged	Disabled by default
Full root shell	su - root (needs root pw)	sudo -i (uses your pw)

Either group works on modern systems — pick one per host for consistency. Add a user to it cautiously.

Beyond sudo: polkit (Awareness)

- **polkit** (PolicyKit) delegates specific **actions** rather than commands.
- Unlike sudo, it doesn't grant full root — it authorizes a defined action (e.g., update software but not install).
- Commands: `pkexec` (run an action), `pkaction`, `pkcheck`.
- Rules are written in JavaScript; actions are defined in XML policy files.

Awareness level for this course — know what polkit is and that pkexec elevates a single action.

Troubleshooting Privilege Escalation

1. **su seems wrong — whoami says root but you're still in your home dir.**

Likely the dash spacing: use `su - root` (space on both sides).

2. **sudo broke after editing /etc/sudoers with nano.**

Reopen with `visudo`; it flags the syntax error nano didn't catch.

3. **A user can't do tasks they're authorized for.**

They're probably not in `wheel/sudo` or not delegated in `sudoers`.

4. **A `sudo/pkexec` command from a website fails.**

The privilege was never delegated via `sudo` or `polkit`.

Security Focus: Least Privilege



SECURITY FOCUS

This unit's security thread is one idea applied everywhere. **Don't work as root** — log in as an ordinary user and elevate only when needed, preferring sudo (your password, logged, per-command). **Guard the admin group** — wheel on Rocky, sudo on Ubuntu — because membership grants system-wide power. **Treat passwords as a control** with chage aging and pam_faillock lockouts. And remember why /etc/shadow exists: expose only what must be exposed.

Why it matters: One careless usermod -G or an unguarded sudoers entry can hand control of the whole system to the wrong person.

Hands-On Lab: Users & Groups

NETLAB+ · start from the WK03_USERS snapshot · both distributions

- Create and modify user accounts on Rocky Linux 10 and Ubuntu Server 24.04.
- Build groups and add members correctly with `usermod -aG`.
- Set password aging with `chage` and verify with `chage -l`.
- Delegate a command through `sudo` using `visudo`, then test it.
- Confirm the `wheel` (Rocky) vs `sudo` (Ubuntu) administrative group on each host.



Graded against observable outcomes on the live systems — a working configuration, not a screenshot.

Key Takeaways



Linux identifies accounts by UID; `/etc/passwd` and `/etc/shadow` define them.



Groups grant access at scale — and `usermod -aG` needs the `-a`.



Shells, variables, aliases, and `chage` shape and secure the user environment.



Work as a standard user; elevate deliberately with `sudo` over `su`.



`wheel` (Rocky) vs `sudo` (Ubuntu) is the admin-group divergence to remember.

MID-STATE TECHNICAL COLLEGE · IT PROGRAMS

Questions?

Unit 3 · Users, Groups, and Authentication

